

Web Technology



React – States, Hooks, and useState

Subin Sahayam, Assistant Professor,

Department of Computer Science and Engineering
Shiv Nadar University

Front End Development Tasks

App state	Data definition, organization, and storage
User actions	Event handlers respond to user actions
Templates	Design and render HTML templates
Routing	Resolve URLs
Data fetching	Interact with server(s) through APIs and AJAX

This topic introduces how React manages and updates data within components.

Templates

- **Blueprint for user interface.**
 - **Reusable Structures – JSX**
 - **Components and Props**
 - **Eg: Navbar, Product List**
- **Front Task**
 - **Re-use Components to Build UI**
 - **Props – Display Different Data - Same Component**
 - **Modular, Maintainable, and Reusable Code**

Templates

- **Blueprint for user interface.**
 - **Reusable Structures – JSX**
 - **Components and Props**
 - **Eg: Navbar, Product List**
- **Front Task**
 - **Re-use Components to Build UI**
 - **Props – Display Different Data - Same Component**
 - **Modular, Maintainable, and Reusable Code**

1. Blueprint for User Interface

Reusable Structures – JSX

JSX lets us write HTML-like code inside JavaScript.



Components and Props

Components are reusable building blocks. Props pass data to components.



Eg: Navbar, Product List

Common UI parts like Navbar and Product List can be reused across pages.



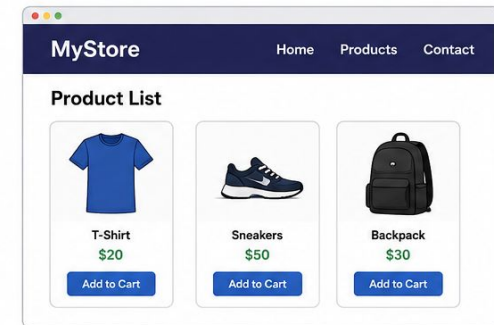
Example: JSX Structure

```
const Navbar = (props) => {
  return (
    <nav className="navbar">
      <h1>{props.title}</h1>
      <ul>
        <li>Home</li>
        <li>Products</li>
        <li>Contact</li>
      </ul>
    </nav>
  );
};
```



Renders UI like

UI Example



2. Front Task



Re-use Components to Build UI
Build the entire interface by combining small, reusable components.



Props – Display Different Data - Same Component
Use props to pass different data and display it using the same component.



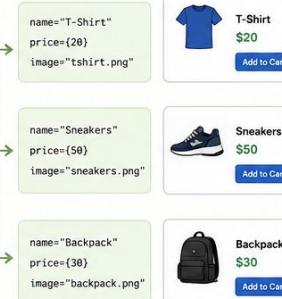
Modular, Maintainable, and Reusable Code
Makes code clean, easy to maintain, and scalable.

Reusing Components with Props

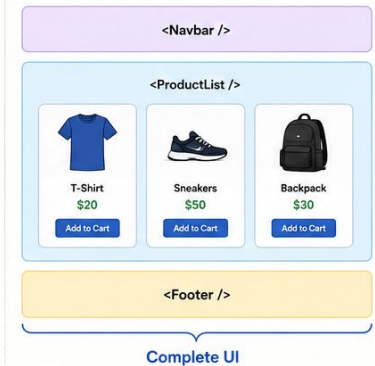
ProductCard Component

```
const ProductCard = ({ name, price, image }) => {
  return (
    <div className="card">
      <img src={image} alt={name} />
      <h3>{name}</h3>
      <p className="price">${price}</p>
      <button>Add to Cart</button>
    </div>
  );
};
```

Same Component
Different Props



Building UI by Combining Components



React State and Lifecycle

1. **State**
2. **Lifecycle**

React State and Lifecycle

1. State

- **Memory**
- **Built-in Object – Stores Data (a.k.a Variable)**
- **State Change -> Component Re-renders + Updates UI**

2. Lifecycle

React State and Lifecycle

1. State

- **Memory**
- **Built-in Object – Stores Data (a.k.a Variable)**
- **State Change -> Component Re-renders + Updates UI**
- **Props are not States**
 - **Props are Read only – States are Mutable**
 - **Props passed from Parent to Children – States Managed Within Component**
 - **Each Component – Independent State**

2. Lifecycle

Mount: Component enters the UI

Update: Component changes

Unmount: Component leaves the UI

React State and Lifecycle

1. State = Component's Memory

```
function Counter() {  
  const [count, setCount] = React.useState(0);  
  return {count};  
}
```

For example:

You can think: State = information that a component remembers while it is running.

For example:

- Counter → remembers the count
- Form → remembers what the user typed
- Login page → remembers whether the user is logged in
- Shopping cart → remembers selected items

React State and Lifecycle

2. State is like a variable

Normally in JavaScript we can write:

```
let count = 0;
```

But React needs to **know when the value changes**, so that it can update the webpage.
Therefore, React provides:

```
const [count, setCount] = React.useState(0);
```

Here:

count → current value

setCount → function to change the value

0 → initial value

So `useState()` gives the component a special kind of variable that React can monitor.

React State and Lifecycle

3. State Change → Component Re-renders

This is one of the most important concepts.

Suppose:

```
const [count, setCount] = React.useState(0);
```

Initially:

```
count = 0
```

The browser displays: Count: 0

When we do: `setCount(1);`

React notices:

State changed



React re-renders component



UI gets updated



Count: 1

So: Changing state tells React: "The data has changed; update the UI."

React Hooks

1. Hooks?

Hooks are built-in React functions that allow functional components to use React features such as state and lifecycle-related functionality.

React Hooks

1. Hooks – Enables

- States
- Lifecycle

React Hooks

- 1. Hooks – Enables**
 - **States**
 - **Lifecycle**
- 2. Eliminates need for class component**

React Hooks

1. Hooks – Enables

- States
- Lifecycle

2. Eliminates need for class component

3. Types of Hooks

- `useState`
- `UseEffect`, `useContext`
- `useReducer`
- `useMemo`
- `useRef`
- `useCallback`

```
class Counter extends React.Component {  
  constructor() {  
    super();  
    this.state = { count: 0 };  
  }  
  
  render() {  
    return (  
      <button onClick={() => this.setState({  
count: this.state.count + 1 })}>  
        {this.state.count}  
      </button>  
    );  
  }  
}
```

with hook

```
function Counter() {  
  const [count, setCount] =  
    React.useState(0);  
  
  return (  
    <button onClick={() =>  
setCount(count + 1)}>  
      {count}  
    </button>  
  );  
}
```

React Hooks - useState

1. Store State Values
2. React Re-renders When Value Changes
3. Syntax: [**<variable>**, **<function>**] = useState(value);
4. `import {useState} from react;`

```
const [val, setVal] = useState(100);
```



The current value



A setter function to
change the value



The initial
value to use

React Hooks - useState

1. Steps

- Import {useState}

```
const [val, setVal] = useState(100);
```



The current value



A setter function to
change the value



The initial
value to use

React Hooks - useState

1. Steps

- Import {useState}
- Set the Initial State Variable and Setter Function

```
const [val, setVal] = useState(100);
```

The current value

A setter function to
change the value

The initial
value to use

React Hooks - useState

1. Steps

- Import {useState}
- Set the Initial State Variable and Setter Function
- Define State Change Logic

```
const [val, setVal] = useState(100);
```



The current value



A setter function to
change the value



The initial
value to use

React Hooks - useState

1. Steps

- Import {useState}
- Set the Initial State Variable and Setter Function
- Define State Change Logic
- Test State Change Logic

```
const [val, setVal] = useState(100);
```

The current value

A setter function to
change the value

The initial
value to use

React Hooks – Online Sources

1. State Hooks

- useState – React

Front End Development Tasks

App state	Data definition, organization, and storage
User actions	Event handlers respond to user actions
Templates	Design and render HTML templates
Routing	Resolve URLs
Data fetching	Interact with server(s) through APIs and AJAX

App State

1. **Data that Determines UI (Essentially States)**
 - **Local Component State Management – useState**
 - **Global State Management – useContext, Redux, and Zustand**

App State

1. **Data that Determines UI (Essentially States)**
 - **Local Component State Management – useState**
 - **Global State Management – useContext, Redux, and Zustand**
2. **Front End Task**
 - **Keeping Count**
 - **Tracking User Input in Forms**
 - **Controlling UI based on Authentication Status**
 - **Managing which Modal or Tab is Open**

User Actions

1. UI Reacts to User Events

- Clicks, Typing, or Form Submissions using Event Handlers
- `onClick`, `onChange`

User Actions

1. UI Reacts to User Events

- Clicks, Typing, or Form Submissions using Event Handlers
- `onClick`, `onChange`

2. Front End Task

- Handling button clicks to submit data or toggle views
- Capturing and updating form input values
- Navigating between pages or sections based on user interaction

References

1. **Robin Wieruch, “The Road to React: The ReactJS with Hooks”, Lean Publishing, 2023 Edition.**
2. **Slides – Aaron Smith - University of North Carolina, Chapel Hill**

THANK YOU

SHIV NADAR
— UNIVERSITY —
CHENNAI